Advances in Symbolic Probabilistic Model Checking with PRISM

Joachim Klein(✉), Christel Baier, Philipp Chrszon, Marcus Daum, Clemens Dubslaff, Sascha Klüppelholz, Steffen Märcker, and David Müller

Institute of Theoretical Computer Science, Technische Universität Dresden, 01062 Dresden, Germany
klein@tcs.inf.tu-dresden.de

Abstract. For modeling and reasoning about complex systems, symbolic methods provide a prominent way to tackle the state explosion problem. It is well known that for symbolic approaches based on binary decision diagrams (BDD), the ordering of BDD variables plays a crucial role for compact representations and efficient computations. We have extended the popular probabilistic model checker PRISM with support for automatic variable reordering in its multi-terminal-BDD-based engines and report on benchmark results. Our extensions additionally allow the user to manually control the variable ordering at a finer-grained level. Furthermore, we present our implementation of the symbolic computation of quantiles and support for multi-reward-bounded properties, automata specifications and accepting end component computations for Streett conditions.

1 Introduction

One prominent approach to cope with the well-known state-explosion problem in model checking is the use of symbolic methods based on binary decision diagrams (BDDs) [8,31]. Various BDD-variants have been studied and implemented in tools for the quantitative analysis of probabilistic systems, see, e.g., [4,10,17–19,23,28,32,34]. The prominent probabilistic model-checker PRISM [26,34,35] uses symbolic approaches relying on a multi-terminal binary decision diagram (MTBDD) [3,15] representation of the model. Among others, PRISM provides support for modeling and the analysis of discrete-time Markov chains (DTMC) and Markov decision processes (MDP) as well as continuous-time Markov chains (CTMC) against temporal logical specifications. While the behavior of Markov chains is purely probabilistic, MDPs exhibit both probabilistic and nondeterministic choices. The typical task of the analysis of MDPs is to compute a scheduler for resolving the nondeterminism that maximizes or minimizes the probability for a given path property or an expectation. The symbolic implementation of

The authors are supported by the DFG through the collaborative research entre HAEC (SFB 912), the Excellence Initiative by the German Federal and State Governments (cluster of excellence cfAED and Institutional Strategy), the Research Training Groups QuantLA (GRK 1763) and RoSI (GRK 1907), the DFG/NWO-project ROCKS, and Deutsche Telekom Stiftung.

© Springer-Verlag Berlin Heidelberg 2016
M. Chechik and J.-F. Raskin (Eds.): TACAS 2016, LNCS 9636, pp. 349–366, 2016.
DOI: 10.1007/978-3-662-49674-9_20

PRISM comes in three flavors: a purely symbolic engine MTBDD and two semi-symbolic engines, called HYBRID and SPARSE. The MTBDD engine performs all computations using MTBDDs, while the HYBRID engine combines the MTBDD-based representation of the transition matrix of the model with an explicit representation of the solution state value vector in the computation [24]. The latter is motivated by the observation that the MTBDD representation of such state value vectors can be of substantive size even for models with compact MTBDD representation during probabilistic model-checking algorithms. The SPARSE engine constructs an explicit, sparse matrix from the MTBDD-based transition matrix for numerical computations and performs computations using this explicit representation. In addition to the three symbolic engines, which rely internally on the infrastructure of the C-based CUDD library [37] for MTBDD storage and manipulation, the fourth engine, EXPLICIT, is fully implemented in Java, builds an explicit representation of the reachable state space of the model and carries out all analysis on this explicit representation. Depending on the concrete model structure and size, each of the four engines has situations where it can show its particular strength.

It is well known that the variable order of the BDD variables plays a crucial role for obtaining a compact representation of the model and for model checking performance. PRISM provides limited influence on the variable order, mainly during modeling by the order in which individual modules are placed in the model file and the order of the individual state variables inside a module. While care has been taken to use a sensible variable order derived from the structure of elements in the model file [34], PRISM lacks any support for automatically identifying a good variable order using techniques such as sifting [33,36], which are routinely employed in symbolic model checkers for non-probabilistic systems (e.g., [11]). In our previous work on complex case studies, we have reached several times the point where we had to resort to manually swapping the module and variable definitions in the model file to try and find a better ordering, in particular for models where explicit approaches were infeasible (e.g., [13]).

Contribution. The main purpose of this paper is to present several refinements of PRISM's symbolic engines. First, we added support for the *automated variable reordering* of the MTBDD-based model representation by enabling CUDD's implementation of group sifting and by extensions of PRISM's input modeling language that allow to rearrange and interleave the orders of the bits of state variables within the same module as well as (the bits) of state variables of different modules. The impact of the automated reordering has been evaluated using the examples from the PRISM benchmark suite [27] and in the context of the symbolic quantile computations. Our second contribution are symbolic implementations of computation schemes for *cost-* or *reward-bounded reachability properties* in discrete Markovian models (DTMCs or MDPs) and corresponding *quantiles*[1]. The latter are, e.g., useful to compute the minimal energy budget required to

[1] While PRISM supports the computation of expected costs or rewards and probabilities for step-bounded properties, it does not contain implementations of algorithms for computing probabilities for reachability conditions with cost/reward constraints.

ensure a 90 % chance for completing a list of jobs. Algorithms for the computation of quantiles have been presented in [5,38] and prototypically implemented using (non-symbolic) explicit representations of the model. Within this paper, we report on the results of comparative experimental studies of the explicit and the new symbolic implementation. The third contribution are enhancements of PRISM's engines for the *automata-based analysis* of DTMCs and MDPs. This includes the treatment of Streett acceptance conditions in MDPs (PRISM only offers engines for Rabin acceptance and its generalized variant) and an extension of PRISM's property syntax for automata-specifications (rather than LTL-specifications).

Outline. Section 2 presents our new approaches for variable reordering in PRISM. Section 3 summarizes the main features of our implementations for cost/reward-bounded properties and quantiles, while Sect. 4 presents the automata-based extensions. For further details (implementation, experiments) and an extended version [21] see http://wwwtcs.inf.tu-dresden.de/ALGI/PUB/TACAS16/. We are collaborating with PRISM's authors to integrate our extensions into the main PRISM version and would like to thank David Parker for fruitful discussions.

2 Automatic Variable Reordering in PRISM

Here, we will briefly describe the relevant infrastructure in PRISM for dealing with variable ordering. The MTBDD variable ordering of the symbolic model representation is determined by the order of module and variable definitions in

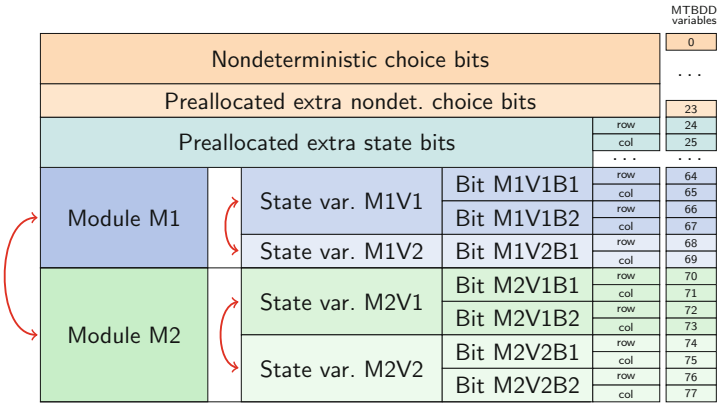

Fig. 1. Schema for the standard variable ordering used by PRISM. The arrows indicate the effect of syntactic reordering in the PRISM model file on the variable order.

the PRISM model file. Figure 1 sketches the general schema[2]. In a first block, MTBDD variables for nondeterministic choices are allocated. This includes a unary encoding of the synchronizing actions (i.e., one MTBDD variable for each action), scheduling variables (one MTBDD variable indicating that a given module is active) as well as several bits for representing local choices, e.g., between alternative commands for the same synchronizing action. Then, two blocks of extra variables are preallocated to serve in later model transformations, e.g., during a product construction with a deterministic ω-automaton for LTL model checking. For each individual bit of a state variable in the model, two MTBDD variables are allocated, one serving in the representation of the rows and one for the columns of the transition matrix. The MTBDD variables for representing the possible values of the (integer-valued) state variables are allocated in the order in which they appear in the PRISM model file, with each state variable forming a block of row/column pairs. The bits for each state variable are ordered from most-significant to least-significant. Global state variables are treated as if they were contained in a single module located before the "real" modules.

The arrows in Fig. 1 indicate the extent of the influence that can be applied to the variable ordering by syntactically reordering the PRISM source file: At the highest level, the order of the modules can be changed. Additionally, inside each module, the order of the definition of the state variables can be changed. Note that such changes of the ordering in the PRISM model file do not lead to any semantic changes in the model, but can lead to cosmetic changes, e.g., in the order of the states for exported models. To complement the manual, trial-and-error approach for finding a good order in the model file, we detail our automatic approach in the next section.

2.1 Automatic Variable Reordering Using Group Sifting

PRISM internally relies on the CUDD (MT)BDD library [37] for the management of a set of BDDs that arise during probabilistic model checking. CUDD provides implementations of several heuristics for (dynamic) variable reordering which in principle should be available to be used by PRISM. Unfortunately, the implementation of PRISM heavily relies on the assumption that the variable ordering of the MTBDD does not change at all. The order of the MTBDD variables is assumed to correspond with the order of the respective variables in the underlying PRISM model, i.e., that the variable index (logical index) and the variable level (index in the current variable order) need to agree. Eliminating this restriction on the variable order would require a substantial refactoring of PRISM's infrastructure, touching many parts of the implementation.

[2] The depicted scheme corresponds to the default ordering for the Hybrid and Sparse engines. There are subtle differences when using the Mtbdd engine, for a detailed discussion see [21]. Additionally, standard PRISM preallocates only extra *state* variables, mainly for the product with deterministic automata. To support generic symbolic model transformations, we also preallocate choice variables, i.e., for fresh actions in the transformed MDP.

Our approach presented in this section makes automatic variable reordering available to a PRISM user while avoiding any substantial refactoring of PRISM's infrastructure. First, a symbolic, MTBDD-based representation of the model is built by PRISM as usual. After the model is built, we trigger the group sifting reordering heuristic [33,36] via the CUDD library, using several variable grouping constraints that will be detailed later. After this reordering, the MTBDD-based model representation violates PRISM's assumptions, which renders further computations in PRISM impossible. Thus, we perform an analysis of the variable ordering found by group sifting and translate the changes in variable locations back to the source level of the PRISM model. This way, we obtain a syntactically reordered PRISM model, where the placement of the PRISM modules und state variables reflects the calculated variable ordering. Our implementation then allows using this reordered model directly after the reordering computation via the following trick: After reordering, we delete the MTBDDs of the model and reset the variable ordering in CUDD to the one that PRISM expects, where each variable index corresponds to the variable level in the BDD. Then, we build the BDDs for the model a second time, this time using the syntactically reordered PRISM model. We thus obtain the reordered model again, but now with the underlying assumptions of PRISM intact, allowing to use the full PRISM machinery. This approach provides transparent and convenient access to the reordering functionality to the user. Additionally, we also support exporting the reordered model to a file, which can then be used in future PRISM runs. This way, the time for reordering can be amortized over multiple model-checking runs.

For this approach to work, it is crucial that we are able to seamlessly convert between the reordered variable ordering obtained after sifting and the variable order that is induced by syntactically reordering the elements of the PRISM model file. To achieve this, we introduce appropriate groups of MTBDD variables represented by a tree structure and used in the groups sifting. The grouping reflects the structure of the given model file: Each PRISM module forms a group of BDD variables that can be reordered as a block. This corresponds to syntactically changing the order of modules in the model file. Additionally, inside each module, the MTBDD variables for each state variable form another group. Reordering those groups corresponds to changing the order of the variable declarations inside a PRISM module. The remaining variables, e.g., those for nondeterministic choices remain in fixed positions. Hence, the above approach allows for creating all variable orders that can result from permutations of modules and state variables within the PRISM model file. In the next section we show how a more fine-grained control can be achieved.

2.2 Bit-Level Control over the Variable Order Using Views

Although it is well known that for some operators, e.g., the addition of two integers, an efficient representation relies on the interleaving of the individual bit-variables, there is no way of interleaving the individual bits of multiple state variables in PRISM up to now.

```
module M
   s_bit_2 : [0..1];
   s_bit_1 : [0..1];
   s_bit_0 : [0..1];

   s : view (s_bit_2,s_bit_1,s_bit_0) <=> [2..7] init 3;

   [inc] s<7 -> 1:(s'=s+1);
endmodule
```

Fig. 2. Defining a view `s` with data domain $(2, 7)$ from three single-bit state variables.

Our implementation provides the option of syntactically "exploding the bits" of all the state variables in a PRISM model file: Each multi-bit state variable s is replaced with the appropriate number of single-bit variables s_i. To keep this transformation simple and transparent to the user we introduce a syntactic enhancement of the PRISM modeling language called a *view*. A view forms a virtual variable s over bit variables s_j. This virtual variable can be used in guards and updates of transition definitions just as ordinary variables. Hence, exploding the bits does not affect any of the transition definitions given in the model file.

As an example, consider the PRISM module in Fig. 2. Here, the virtual state variable `s` with an integer data domain of $2 \leqslant \mathtt{s} \leqslant 7$ requires three bits to represent all values, as internally integer variables are encoded by first subtracting the lower bound of the data domain (2 is internally represented as 0, etc.). The actual storage is provided by the three single-bit state variables `s_bit_`i. The order of the single-bit state variables in the view definition determines their use in the encoding, with the most-significant bit appearing first. As can be seen, the virtual view variable `s` is being used just like a standard PRISM state variable.

Note that "exploding the bits" of a PRISM model file alone will not change the variable ordering and MTBDD representation, as the encoding and ordering of the newly introduced single-bit state variables correspond to the standard encoding used for the original variables. When applying the automatic reordering detailed in the previous section to an "exploded" model file, the individual bits of the state variables can be now be sifted and interleaved, as their grouping is removed. However, the MTBDD variables are still restricted from crossing module boundaries. We detail how to remove this restriction in the next section.

2.3 Interleaving State Variables of Different Modules

To overcome the limitation that state variables cannot be interleaved across modules our implementation provides the option of "globalizing" all state variables in a PRISM model file: Each state variable inside a PRISM module is moved from the module to become a global variable, while keeping the order they appeared in the original model file. Realizing this requires to loosen some restrictions on the use of global variables imposed by PRISM. In standard PRISM, global variables cannot be updated in synchronous actions, as this has the potential of resulting in conflicting updates from multiple modules. We removed this restriction, as in

```
module M1
  x : [0..3] init 0;
 [a] true ->   0.5:(x'=0)
             + 0.5:(x'=y);
endmodule

module M2
  y : [0..3] init 0;
 [a] true -> 1:(y'=0);
 [b] y<3  -> 1:(y'=y+1);
endmodule
```

```
global x_bit_1 : [0..1];
global x_bit_0 : [0..1];
global y_bit_1 : [0..1];
global y_bit_0 : [0..1];
global x :
  view (x_bit_1,x_bit_0) <=> [0..3] init 0;
global y :
  view (y_bit_1,y_bit_0) <=> [0..3] init 0;

module M1
  [a] true -> 0.5:(x'=0) + 0.5:(x'=y);
endmodule

module M2
 [a] true -> 1:(y'=0);
 [b] y<3  -> 1:(y'=y+1);
endmodule
```

Fig. 3. Example of both "exploding bits" and "globalizing variables" for a PRISM model file (before on the left, after on the right).

our setting only the "previous owner" of a variable, i.e., the module in which the variable was initially declared, will update the global variable in the transformed model. This ensures that there can be no conflicting updates introduced by globalizing variables. Our implementation supports such global variable updates for similar situations as well, i.e., where it is apparent by a syntactic inspection that no conflicting updates can happen.

The options for exploding the bits and globalizing the variables can be used separately and in a combined fashion (cf. Fig. 3) and the resulting model yields a starting point for group sifting. This way, fine-grained control of the variable ordering for all state variables in the model becomes possible. Within the following section we will evaluate our implementation by means of a number of case studies.

2.4 Benchmarking Automatic Variable Reordering of PRISM Models

To explore the effect of automatic variable reordering using our implementation, we performed benchmarks using the DTMC, CTMC and MDP models in the PRISM benchmark suite [27]. The models are parametrized in various parameters, affecting both the number of states and the size of the MTBDD representation. In total, we performed benchmarks with 208 model instances (70 DTMCs, 70 CTMCs, 68 MDPs). We present here statistics for the "top" initial variable ordering [34] used by default in the Hybrid engine. Results using the default variable ordering of the Mtbdd engine were roughly similar.

Figure 4 presents statistics for the basic case, i.e., reordering without any syntactic transformations beforehand. Similar plots for reordering with the "globalize variables" (Sect. 2.3) and "explode bits" (Sect. 2.2) transformations being

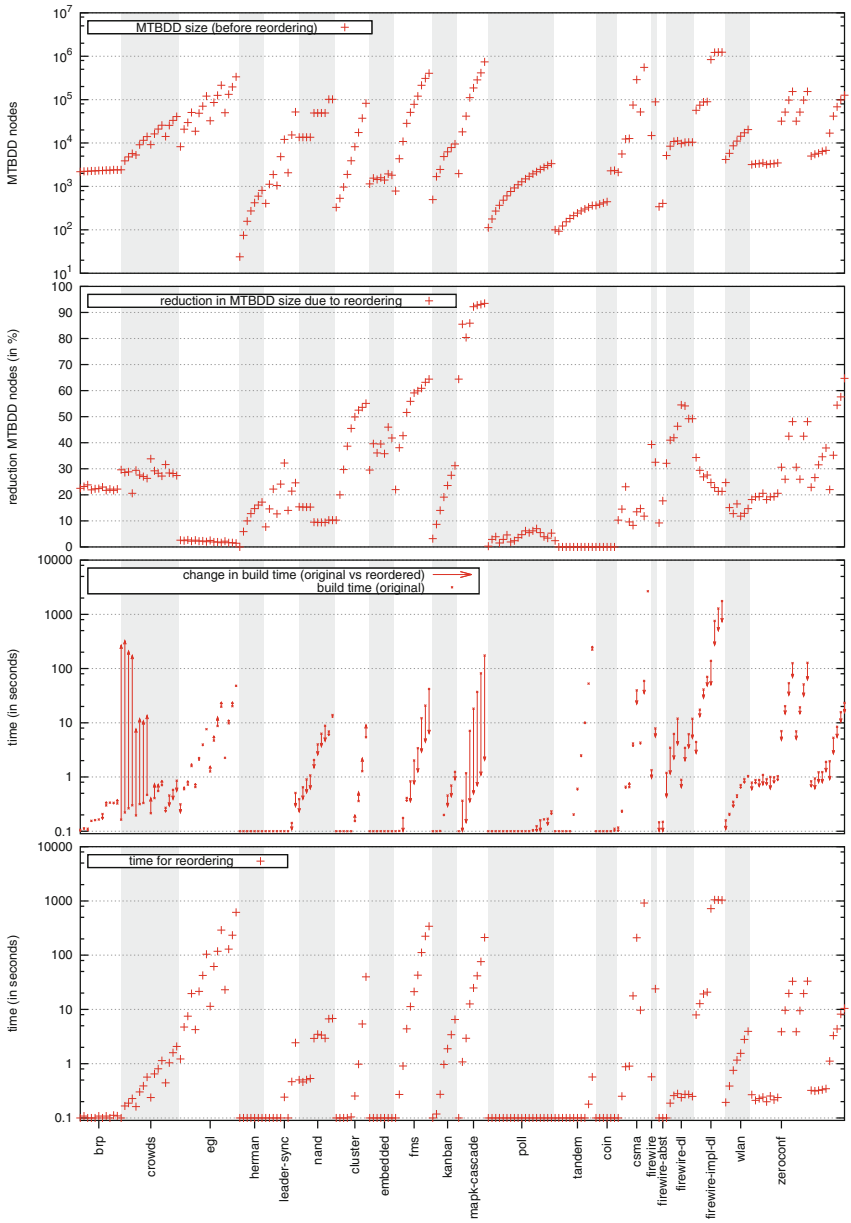

Fig. 4. Statistics for reordering without syntactic transformations: The number of MTBDD nodes before reordering, the reduction (larger numbers represent more reduction) in the number of MTBDD nodes, the change in time for building the model (before/after reordering) and the time spent reordering. Times below 0.1 s are clipped to 0.1 for visualization purposes. There was one timeout, reordering the "csma4_6" instance (45 min of the 1 h timeout spent on building, with 3,589,198 nodes).

applied can be found in [21][3]. In the plots, the model instances are grouped by their base model. The size of the MTBDD refers to the number of nodes in the shared MTBDD structure storing the various individual MTBDDs. Those individual MTBDDs represent the model in PRISM, i.e., its transition matrix, a 0/1-version of the transition matrix representing the underlying graph structure of the model, the set of reachable states, representations for the transition and state rewards.

As can be seen in the second plot from the top in Fig. 4, the automatic reordering was able to achieve significant reductions for many of the model instances. As a particularly striking example, the reordering was very effective for the "mapk-cascade" model, a CTMC: For the instance with parameter $N = 8$, the MTBDD size was reduced from 1,478,511 nodes to 96,718 nodes, a reduction of more than 90 %. The time for building the symbolic representation of this model instance was reduced from 174 s to 2 s for the reordered model. Most of the time, the reduction in the MTBDD size is accompanied by a reduction in the time needed for building the MTBDDs for the reordered model. The major outlier to this are several instances of the "crowds" model, where the time for building the reordered model was substantially worse compared with the original model. Our investigation revealed that this is due to the point in time at which our reordering is performed, i.e., after the symbolic transition matrix has been restricted to the reachable part of the state space, which is the symbolic representation that is then used for the actual model checking. The reordering heuristic thus produced a variable order tailored for this state space and which is not particularly suitable for the representation of the individual, not yet restricted parts of the model used during the building phase. This is a classic example of the case where an asynchronous reordering, i.e., continuously adapting the variable ordering during the construction phase, would be helpful.

In general, the time for reordering tends to be related to the size of the MTBDD before reordering, as expected. As noted above, even substantial reordering times might be worthwhile, as the reordered model can be stored and subsequently reused multiple times, profiting, e.g., from the reduced build time and more compact symbolic representation.

There were three models ("brp", "nand" and "poll"), where instances exhibited an overall reduction in the size of the MTBDD, but an increase in the size of the MTBDD for the transition matrix alone (in all cases the increase was less than 10 %). This is explained by the fact that the reordering operates on the whole shared MTBDD data structure and thus does not necessarily optimize all the individual MTBDD functions that are stored.

We have also benchmarked the effect of our syntactic transformations on the automatic reordering and present here (Table 1) some notable examples. For further, detailed statistics we refer to [21]. As already seen in Fig. 4, the "tandem"

[3] The benchmarks for reordering were carried out on a machine with two Intel Xeon L5630 4-core CPUs at 2.13 GHz and 192 GB RAM, with a timeout of 1 h and a CUDD memory limit of 10 GB. The max-growth factor of CUDD was set to 2, i.e., allowing a doubling in MTBDD size before sifting is abandoned.

Table 1. Selected statistics for the reduction achieved using reordering on the standard model instance and where the "explode bits" and "globalize variables" transformations were applied. In the last column, both transformations are applied. For reference, the MTBDD size before reordering is included as well. For full details, see [21].

Model	instance	MTBDD before	reduction in %			
			standard	explode	globalize	exp.+glob.
tandem	c=255	4 917	0.0	26.0	0.0	35.1
tandem	c=4095	103 233	0.0	35.7	0.0	64.3
cluster	N=32	7 391	45.5	47.9	52.2	8.3
cluster	N=256	61 749	53.6	58.7	24.2	41.4
cluster	N=512	132 908	55.1	59.3	61.5	46.2
kanban	t=6	14 001	27.5	34.0	1.2	32.3

model has no reduction in MTBDD size when it is reordered as-is. However, when the state variables are "exploded", reordering becomes profitable, with additional reductions when combined with the "globalize variables" transformations. Globally, for every model instance from the benchmark suite, at least one of the variants achieved some reduction. As is to be expected, no variant is uniformly best. Consider the statistics for the "cluster" model in Table 1. For $N = 32$, "exploding" and "globalizing" are individually successful, but in combination lead to only minor reductions. For $N = 256$, "exploding" is in the lead, while for $N = 512$, "globalizing" by itself leads to the most reductions. For "kanban" with $t = 6$, "globalizing" alone leads to worse reductions than reordering on the standard model. As can be seen, it remains an area of experimentation to select the reordering variant that is a good fit for a particular model and model instance. As a good first assumption, the time for model checking tends to be related in general to the compactness of the symbolic representation. We experimented with some of the properties in the benchmark suite (cf. [21] for some examples). In the next section, we will additionally report on significant reductions in model-checking time in the context of quantile computations with reordered models.

3 Computing Quantiles for Markov Decision Processes

Models in PRISM can be annotated with rewards (non-negative values) specifying the costs or the gain for visiting certain states or taking certain transitions. PRISM provides implementations of algorithms for reasoning about expected rewards, but lacks support for computing the probabilities for reward-bounded path properties, unless for unit-reward functions that count the number of steps. We have extended PRISM with support for the computation of (extremal) probabilities of cost-/reward-bounded simple path formulas for DTMCs and MDPs with non-negative integer rewards, e.g., of $\Pr^{\max}(\Diamond^{\leq r}\, \Phi)$ for a reward bound r.

This includes conjunctions of multiple reward bounds and step bounds [1], relying on a product transformation with a counter automaton tracking the accumulated reward. This is implemented for both the explicit and symbolic engines.

In our recent work [5,38], we addressed the computation of quantiles for probability constraints on reward-bounded reachability conditions and carried out experiments with a prototypical implementation based on PRISM's EXPLICIT engine. In the mean time, this implementation has been refined and extended by a symbolic implementation. In what follows, we describe some details of the latter. We consider here MDP with a reward function $rew\colon S \times Act \to \mathbb{N}_{\geqslant 0}$, mapping state-action pairs (s, α) to the non-negative integer reward $rew(s, \alpha)$. The quantiles under consideration (for details we refer to [5]) are optimal reward thresholds that guarantee that the maximal or minimal probability of a reward-bounded reachability path formula meets some probability bound. Examples are $\min\{\, r \,:\, \Pr^{\max}(\Diamond^{\leq r}\, \Phi) > p \,\}$ or $\max\{\, r \,:\, \Pr^{\min}(\Diamond^{\geq r}\, \Phi) > p \,\}$ where r can be seen as a parametric reward bound, Φ is a state formula and p a rational probability bound. Quantiles yield a useful concept for the cost-utility analysis, e.g., in terms of the minimal amount of energy r required to reach some goal Φ with probability at least p for some/all schedulers. The approach for computing quantiles as proposed in [5] consists of a two-step process. A precomputation step determines all states $s \in S$ for which the quantile exists, i.e., is finite. In the simplest case, this amounts to the computation of the maximal probability for unbounded reachability. In other cases, the computation requires the analysis of zero-reward and positive-reward end components [5]. For the remaining states, an iterative approach is used, which we illustrate here for a quantile of the form $\min\{\, r \,:\, \Pr^{\max}(\Diamond^{\leq r}\, \Phi) > p \,\}$ where we suppose the MDP has a unique initial state s_0. Successively, the values $x_{s,r} = \Pr_s^{\max}(\Diamond^{\leq r}\, \Phi)$ for $r = 1, 2, 3, \ldots$ are computed for all states $s \in S$ until some r with $x_{s_0,r} > p$ is reached, using the equation $x_{s,r} = \max\{A_s, B_s\}$ with

$$A_s = \max_{\alpha \in Act(s), rew(s,\alpha)=0} \sum\nolimits_{t \in S} \Pr(s, \alpha, t) \cdot x_{t,r}$$

$$B_s = \max_{\alpha \in Act(s), rew(s,\alpha)>0} \sum\nolimits_{t \in S} P(s, \alpha, t) \cdot x_{t, r - rew(s,\alpha)}$$

where $\Pr(s, \alpha, t)$ is the probability of reaching state t when action α is chosen in state s. For states satisfying Φ, $x_{s,r}$ is set to 1 for all r. The values A_s, handling the zero-reward actions, are computed using value iteration. The values B_s, handling the positive-reward actions, are determined by inserting the previously calculated values $x_{t,i}$ for $i < r$. For the other quantile variants, similar computations are performed [5]. The time complexity of this approach is exponential, meeting the complexity-theoretic optimum [16].

3.1 Symbolic Computation of Quantiles

We have extended PRISM with a symbolic implementation for the computation of quantiles, following the general approach outlined above. For the precomputation

step, we rely on the PRISM machinery for the computation of maximal/minimal probabilities for unbounded path formulas and for the computation of (maximal) end components, adapted for identifying states in positive-reward end components and zero-reward end components by appropriate symbolic model transformations.

For the iterative computation of the values $x_{s,r}$ for $r = 1, 2, 3, \ldots$ until the probability threshold p is reached, the values $x_{s,r}$ are stored symbolically, using one MTBDD per bound r to represent the functions $x_r \colon S \to \mathbb{Q}$. The state-action pairs with positive reward are handled first, computing the MTBDD $B \colon S \to \mathbb{Q}$. Here, all state-action pairs with identical reward value are handled simultaneously. Consequently, this symbolic approach tends to be most efficient if there are many state-action pairs, but few distinct reward values in the model. To subsequently handle the zero-reward state-action pairs, we symbolically transform the MDP. First, all positive-reward actions are stripped and replaced by a single fresh τ-action for each state. These τ-actions model the choice of choosing the "best" positive-reward action in a state s and go to a special *goal* trap state with probability $B(s)$ and to a *fail* trap state with probability $1 - B(s)$. The computation of $x_{s,i}$ then amounts to a standard maximal/minimal reachability probability computation in the transformed model by means of value iteration, relying on the computation engine chosen by the user, i.e., either the MTBDD, HYBRID or SPARSE engine. As the state value vectors x_r are stored symbolically in all cases and the use of the semi-symbolic techniques of the HYBRID and SPARSE engines is thus limited, we denote these engines as SEMIHYBRID and SEMISPARSE in the context of our symbolic quantile implementation.

3.2 Benchmarks for Quantile Computations

To perform benchmarking of our implementation, we have reused several models and quantile queries that were first considered in [5] for benchmarking our quantile implementation for PRISM's EXPLICIT engine. We present here (Table 2) statistics for some noteworthy model instances, for further statistics and details on the models and quantile queries we refer to [21][4].

As can be seen in Table 2, there are model instances were the quantile implementation in the EXPLICIT engine easily outperforms our symbolic approach, e.g., for the "self-stabilizing" case study, despite a very compact MTBDD representation of the model. To put computation times such as 2153.2 s (for N=18) into context, the number of iterations in the quantile computation has to be kept in mind: Here, 392 iterations were required, with an average time per iteration of around 5 s. The large number of iterations thus amplifies the time spent in each iteration. For the "asynchronous leader-election" case study, our symbolic implementation becomes competitive for N=8 because of the time spent for model construction in the EXPLICIT engine due to the large state space. The symbolic computations still yield results for N=9, where the explicit approach times out. A similar picture is seen for query Q7 of the "energy-aware

[4] The benchmarks for the quantile computations were carried out on a machine with two Intel E5-2680 8-core CPUs at 2.70 GHz with 384 GB of RAM running Linux.

Table 2. Quantile computations for selected case studies, with statistics for the model size (reachable state space, MTBDD size of symbolic transition matrix) and times spent for model building and computing the quantile query (in seconds). The "it." column depicts the number of overall iterations in the quantile computation.

N	States	MTBDD size	it.	Explicit		symbolic quantile computations					
						SemiHybrid		SemiSparse		Mtbdd	
				t_{build}	t_{query}	t_{build}	t_{query}	t_{build}	t_{query}	t_{build}	t_{query}
Self-stabilizing algorithm (Israeli/Jalfon), N processes (query Q1)											
11	2047	433	144	0.5	0.2	<0.1	2.5	<0.1	2.3	<0.1	2.0
15	32767	729	271	1.6	3.9	<0.1	119.1	<0.1	114.7	<0.1	159.3
18	262143	993	392	9.8	53.8	<0.1	2135.2	<0.1	2865.1	<0.1	2240.3
Asynchronous leader election, N processes (query Q3)											
7	2095783	180383	206	63.1	83.2	5.9	358.8	7.0	369.8	7.6	402.7
8	18674484	392093	238	1633.2	891.8	20.6	1228.4	24.6	1307.4	21.9	1448.8
9	167748115	868257	279	–	–	106.1	7751.9	92.3	5728.1	92.9	7545.6
Energy-aware job scheduling, N processes (query Q7)											
5	6079533	187458	302	334.1	285.9	7.8	1196.2	7.2	1128.0	7.9	1142.9
6	44072357	507805	416	–	–	21.7	3808.4	22.8	4045.9	25.0	3962.8
Energy-aware job scheduling, N processes (query Q8)											
5	3049471	25363	13	62.5	398.6	0.8	86.0	0.9	74.3	0.9	104.6
6	7901694	38911	15	210.0	2375.5	1.8	390.6	1.8	378.1	1.3	317.1

job scheduling" case study. For the instances of this case study and query Q8 shown in Table 2, the symbolic implementation vastly outperforms the explicit implementation. This is mainly due to the large amount of time spent there for the precomputation step (1971 s for N=6), while the symbolic engines perform this step in around 2 s. This appears to be due to inefficiencies in some of the end component computations in the Explicit engine, which we are currently investigating and working on a potential fix. For the "energy-aware job scheduling" case study, the computations were carried out in a reordered model, which lead to a significant decrease in MTBDD size and computation time. For instance, for (Q7) and N=6 we observed a reduction in the size of the transition matrix of 78.2 % and the quantile computation (SemiHybrid) took 43,832.9 s in the original model instead of 3808.4 s in the reordered model, with similar reductions for Mtbdd and SemiSparse.

Quantiles in Feature-Oriented Systems. In product lines, collections of systems are described through the combination of features. Thus, the systems in a product line usually share a lot of common behaviors, which makes symbolic approaches appealing. Using a family-based approach, i.e., modeling the product line in a single model, in previous work [13] we performed experiments on an energy-aware server product line eServer. There, we illustrated the benefits of symbolic representations in product-line verification and showed that variable orderings have a crucial impact on the analysis performance. However, due to

Table 3. Quantile computations for eServer, with statistics for the reachable state space and MTBDD size of the transition matrix, reduction and time for reordering, and time building the model and computing the quantile query (in seconds).

		States	MTBDD nodes	Reduction in %	t_{reorder}	Mtbdd t_{build}	Mtbdd t_{query}
Instance 1	original	145 984 112	64 030	-	-	10.2	1 018.6
	reordered	"	40 096	37.4	3.5	7.8	766.4
	with explode	"	34 902	45.5	8.3	9.1	726.2
	with globalize	"	36 149	43.5	2.9	6.3	704.3
	with exp.+glob.	"	30 325	52.6	3.1	7.9	638.6
Instance 2	original	441 704 832	140 556	-	-	27.2	3 664.6
	reordered	"	72 565	48.4	68.5	16.6	3 510.5
	with explode	"	66 874	52.4	32.3	17.7	3 204.7
	with globalize	"	43 249	69.2	6.5	11.2	3 099.0
	with exp.+glob.	"	37 674	73.3	8.5	12.1	2 833.0

the lack of a symbolic quantile implementation, an energy-utility analysis of eServer had to be postponed as future work.

In Table 3, we summarize statistics for the computation of quantiles on two instances of eServer, becoming possible due to our symbolic implementation. We computed the minimal amount of energy required to guarantee in 95 % of the cases a certain percentage of the time without any package drop. The table shows the impact of our four reorder mechanisms on the model size and the quantile computation time. We only included the results for the Mtbdd engine, as the other engines struggled with the size of the model and reached a timeout after one day. Within all computations, 1476 quantile iterations were required. Interestingly, although the model presented in [13] already used heuristics to find good initial variable orderings, the fully automatic reorder mechanisms presented here allow for a further significant reduction of the model size and a speedup of the analyses.

4 Additional Enhancements

We report here on additional enhancements we have implemented in PRISM both for the symbolic and explicit engines, related to the support of ω-automata.

Accepting End Component Computations for Streett Conditions. Traditionally, PRISM has relied on an internal implementation of Safra's determinization construction for generating the deterministic Rabin automata used for LTL model checking, e.g., for computing $\Pr^{\max}(\varphi)$ or $\Pr^{\min}(\varphi)$ for an LTL formula φ. Recently, support was added for automata with generalized Rabin acceptance [2,9] to benefit from advances in the construction of small deterministic automata [14,22]. This includes support for calling external tools for the transformation of LTL formulas into deterministic Muller-automata, relying on

the recent Hanoi Omega Automata (HOA) format [2], which supports the concise representation of common acceptance conditions in a generic normal form.

We have extended PRISM's MDP model checking with support for Streett conditions, relying on the recursive algorithm for end-component analysis of [6]. Streett conditions are dual to Rabin conditions and are well suited for the specification of fairness constraints and for conjunctions of properties. They appear as well in the computation of conditional probabilities in MDPs [7]. It is well known, both in theory [29] and in practice [20], that for some languages, deterministic Streett automata can be significantly smaller than Rabin automata.

Extremal Probabilities for Automata Specifications. We have furthermore extended the property syntax of the probability operators in PRISM to allow the use of a HOA-automaton file instead of an LTL formula, providing the full power of ω-regular languages. For DTMC models, the full range of acceptance conditions in the normal form of the HOA format [2] is supported. For MDPs, Rabin, generalized Rabin and Streett conditions are supported. For the computation of $\mathrm{Pr}^{\min}$, which requires the complementation of the language of the automaton, we support Rabin and Streett conditions, exploiting their duality.

5 Conclusion

In this paper, we have demonstrated the potential for automatic variable reordering for symbolic model checking in PRISM, including the benefits of now having fine-grained control over the variable order using our syntactic transformations. We have also shown that our symbolic implementation for quantiles is useful in practice, particularly where explicit representations of the model are infeasible.

Future Work: In the area of automatic variable reordering, it would be interesting to support more structured reordering: Often, models are obtained from templates with parametrization, e.g., specifying the number of copies of certain modules in the model. By swapping the variables of all copies simultaneously, it might be possible to discover good initial variable orders from instances with few copies and apply these to instances with more copies. This approach would also be interesting when the aim is to apply symmetry reduction [12,25], as all copies would remain symmetrical. While our syntactic transformations provide very fine-grained reordering for the state variables, it would be interesting to have the option of adding back some restrictions or hints for the reordering by annotating the variable declarations in the PRISM model. This would allow to state preferences which variable should be kept together, etc. In this context it would also make sense to revisit previous work on heuristics for good initial variable orderings in PRISM [30], making use of the finer-grained control that is now possible. In addition, our benchmark results serve as an indication that it would be worthwhile to attempt a refactoring of PRISM to remove the variable order assumptions and add support for asynchronous reordering.

For our symbolic quantile computations, it appears worthwhile to consider an iterative implementation that fully exploits the approach of the Hybrid engine,

with a symbolic transition matrix and explicit state value vector storage. This could allow the application of several of the techniques employed by the quantile computations of the Explicit engine to speed-up the computations.

The implementation of the end component computation for Streett conditions could serve as the base for supporting more complex types of fairness conditions via the approach of [6], such as fairness for the scheduling of the modules in a PRISM model. It would also be interesting to perform a detailed experimental evaluation on the use of Streett versus (generalized) Rabin automata for probabilistic model checking in practice.

References

1. Andova, S., Hermanns, H., Katoen, J.-P.: Discrete-time rewards model-checked. In: Larsen, K.G., Niebert, P. (eds.) FORMATS 2003. LNCS, vol. 2791, pp. 88–104. Springer, Heidelberg (2003)
2. Babiak, T., Blahoudek, F., Duret-Lutz, A., Klein, J., Křetínský, J., Müller, D., Parker, D., Strejček, J.: The Hanoi omega-automata format. In: Kroening, D., Păsăreanu, C.S. (eds.) CAV 2015. LNCS, vol. 9206, pp. 479–486. Springer, Heidelberg (2015)
3. Bahar, R.I., Frohm, E.A., Gaona, C.M., Hachtel, G.D., Macii, E., Pardo, A., Somenzi, F.: Algebraic decision diagrams and their applications. Formal Methods Syst. Des. **10**(2/3), 171–206 (1997)
4. Baier, C., Clarke, E.M., Hartonas-Garmhausen, V., Kwiatkowska, M.Z., Ryan, M.: Symbolic model checking for probabilistic processes. In: Degano, P., Gorrieri, R., Marchetti-Spaccamela, A. (eds.) ICALP 1997. LNCS, vol. 1256, pp. 430–440. Springer, Heidelberg (1997)
5. Baier, C., Daum, M., Dubslaff, C., Klein, J., Klüppelholz, S.: Energy-utility quantiles. In: Badger, J.M., Rozier, K.Y. (eds.) NFM 2014. LNCS, vol. 8430, pp. 285–299. Springer, Heidelberg (2014)
6. Baier, C., Groesser, M., Ciesinski, F.: Quantitative analysis under fairness constraints. In: Liu, Z., Ravn, A.P. (eds.) ATVA 2009. LNCS, vol. 5799, pp. 135–150. Springer, Heidelberg (2009)
7. Baier, C., Klein, J., Klüppelholz, S., Märcker, S.: Computing conditional probabilities in Markovian models efficiently. In: Ábrahám, E., Havelund, K. (eds.) TACAS 2014 (ETAPS). LNCS, vol. 8413, pp. 515–530. Springer, Heidelberg (2014)
8. Burch, J.R., Clarke, E.M., McMillan, K.L., Dill, D.L., Hwang, L.J.: Symbolic model checking: 10^{20} states and beyond. Inf. Comput. **98**(2), 142–170 (1992)
9. Chatterjee, K., Gaiser, A., Křetínský, J.: Automata with generalized rabin pairs for probabilistic model checking and LTL synthesis. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 559–575. Springer, Heidelberg (2013)
10. Ciardo, G., Miner, A.S., Wan, M.: Advanced features in SMART: the stochastic model checking analyzer for reliability and timing. SIGMETRICS Perform. Eval. Rev. **36**(4), 58–63 (2009)
11. Cimatti, A., Clarke, E., Giunchiglia, E., Giunchiglia, F., Pistore, M., Roveri, M., Sebastiani, R., Tacchella, A.: NuSMV 2: an opensource tool for symbolic model checking. In: Brinksma, E., Larsen, K.G. (eds.) CAV 2002. LNCS, vol. 2404, pp. 359–364. Springer, Heidelberg (2002)

12. Donaldson, A.F., Miller, A., Parker, D.: Language-level symmetry reduction for probabilistic model checking. In: Proceedings of the Quantitative Evaluation of Systems (QEST 2009), pp. 289–298. IEEE (2009)
13. Dubslaff, C., Baier, C., Klüppelholz, S.: Probabilistic model checking for feature-oriented systems. Trans. Aspect-Oriented Softw. Dev. **12**, 180–220 (2015)
14. Esparza, J., Křetínský, J.: From LTL to deterministic automata: a Safraless compositional approach. In: Biere, A., Bloem, R. (eds.) CAV 2014. LNCS, vol. 8559, pp. 192–208. Springer, Heidelberg (2014)
15. Fujita, M., McGeer, P.C., Yang, J.C.-Y.: Multi-terminal binary decision diagrams: An efficient data structure for matrix representation. Formal Methods Syst. Des. **10**(2/3), 149–169 (1997)
16. Haase, C., Kiefer, S.: The odds of staying on budget. In: Halldórsson, M.M., Iwama, K., Kobayashi, N., Speckmann, B. (eds.) ICALP 2015. LNCS, vol. 9135, pp. 234–246. Springer, Heidelberg (2015)
17. Hachtel, G.D., Macii, E., Pardo, A., Somenzi, F.: Markovian analysis of large finite state machines. IEEE Trans. CAD Integr. Circ., Syst. **15**(12), 1479–1493 (1996)
18. Hartonas-Garmhausen, V., Campos, S., Clarke, E.: ProbVerus: probabilistic symbolic model checking. In: Katoen, J.-P. (ed.) ARTS 1999. LNCS, vol. 1601, pp. 96–110. Springer, Heidelberg (1999)
19. Hermanns, H., Kwiatkowska, M.Z., Norman, G., Parker, D., Siegle, M.: On the use of MTBDDs for performability analysis and verification of stochastic systems. J. Logic Algebr. Program. **56**(1–2), 23–67 (2003)
20. Klein, J., Baier, C.: Experiments with deterministic ω-automata for formulas of linear temporal logic. Theoret. Comput. Sci. **363**(2), 182–195 (2006)
21. Klein, J., Baier, C., Chrszon, P., Daum, M., Dubslaff, C., Klüppelholz, S., Märcker, S., Müller, D.: Advances in symbolic probabilistic model checking with PRISM (extended version) (2016). http://wwwtcs.inf.tu-dresden.de/ALGI/PUB/TACAS16/
22. Komárková, Z., Křetínský, J.: Rabinizer 3: Safraless translation of LTL to small deterministic automata. In: Cassez, F., Raskin, J.-F. (eds.) ATVA 2014. LNCS, vol. 8837, pp. 235–241. Springer, Heidelberg (2014)
23. Kuntz, M., Siegle, M.: CASPA: symbolic model checking of stochastic systems. In: Proceedings of Measuring, Modelling and Evaluation of Computer and Communication Systems (MMB 2006), pp. 465–468. VDE Verlag (2006)
24. Kwiatkowska, M.Z., Norman, G., Parker, D.: Probabilistic symbolic model checking with PRISM: a hybrid approach. Softw. Tools Technol. Transfer **6**(2), 128–142 (2004)
25. Kwiatkowska, M., Norman, G., Parker, D.: Symmetry reduction for probabilistic model checking. In: Ball, T., Jones, R.B. (eds.) CAV 2006. LNCS, vol. 4144, pp. 234–248. Springer, Heidelberg (2006)
26. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) CAV 2011. LNCS, vol. 6806, pp. 585–591. Springer, Heidelberg (2011)
27. Kwiatkowska, M.Z., Norman, G., Parker, D., The PRISM benchmark suite. In: Proceedings of Quantitative Evaluation of Systems (QEST 2012), pp. 203–204. IEEE (2012). https://github.com/prismmodelchecker/prism-benchmarks/
28. Lampka, K.: A symbolic approach to the state graph based analysis of high-level Markov reward models. PhD thesis, Universität Erlangen-Nürnberg (2007)
29. Löding, C.: Optimal bounds for transformations of ω-automata. In: Pandu Rangan, C., Raman, V., Sarukkai, S. (eds.) FST TCS 1999. LNCS, vol. 1738, pp. 97–109. Springer, Heidelberg (1999)

30. Maisonneuve, V.: Automatic heuristic-based generation of MTBDD variable orderings for PRISM models. ENS Cachan & Oxford University, Internship report (2009). http://www.prismmodelchecker.org/papers/vivien-bdds-report.pdf
31. McMillan, K.L.: Symbolic Model Checking. Kluwer, Norwell (1993)
32. Miner, A.S., Parker, D.: Symbolic representations and analysis of large probabilistic systems. In: Baier, C., Haverkort, B.R., Hermanns, H., Katoen, J.-P., Siegle, M. (eds.) Validation of Stochastic Systems. LNCS, vol. 2925, pp. 296–338. Springer, Heidelberg (2004)
33. Panda, S., Somenzi, F.: Who are the variables in your neighborhood. In: Proceedings of the Computer-Aided Design (ICCAD 1995), pp. 74–77. IEEE (1995)
34. Parker, D.: Implementation of Symbolic Model Checking for Probabilistic Systems. PhD thesis, University of Birmingham (2002)
35. PRISM model checker. http://www.prismmodelchecker.org/
36. Rudell, R.: Dynamic variable ordering for ordered binary decision diagrams. In: Proceedings of the Computer-Aided Design (ICCAD 1993), pp. 42–47. IEEE (1993)
37. Somenzi, F.: CUDD: Colorado University decision diagram package. http://vlsi.colorado.edu/~fabio/CUDD/
38. Ummels, M., Baier, C.: Computing quantiles in markov reward models. In: Pfenning, F. (ed.) FOSSACS 2013 (ETAPS 2013). LNCS, vol. 7794, pp. 353–368. Springer, Heidelberg (2013)